VitalFlow V1 — Patient & Appointment UI

Design doc for the patient, insurance, and appointment surfaces. Most of this is already shipped across Slices 1–3 + the insurance/diagnosis follow-ups; this document formalizes the architecture and names the gaps (the calendar week view is the only material new UI).

See also:

- docs/permissions-matrix.md — who sees/does what
- docs/clinical-domain.md — entity shapes
- docs/ui-guidelines.md — visual language

---

1. Principles

1. **Server Components by default.** Every page fetches its data with the Supabase server client (RLS-safe) and renders HTML. Client Components are reserved for genuine interactivity (typeahead, drag-resize, realtime).
2. **Server Actions for writes.** No client-side mutation libraries; forms submit to server actions that call the DB under the authenticated user's RLS context.
3. **One design system.** All UI uses `@vitalflow/ui` — `Button`, `Card`, `FormField`, `Input`, `Select`, `Table*`, `Badge`, `EmptyState`, `PageHeader`, `AppBreadcrumbs`. No bespoke styling.
4. **Three surface groups**, gated by `(app)` route-group layouts:
   - **provider** (`/`, `/patients/*`, `/encounters/*`, `/appointments/*`) — clinicians
   - **admin** (`/admin/*`) — owner/office_admin/biller
   - **patient** (`/my/*`) — patient portal
5. **Role-aware rendering**: permission-gated buttons and sections. The page renders the read-only form of every section that the role can view; write forms appear only when permissions allow.
6. **Empty / loading / error states are explicit**, via `EmptyState`, `LoadingState`, `ErrorState` from `@vitalflow/ui/patterns`. No blank screens.
7. **URL-first state.** Filters (date, status, search query, page) live in the query string, not client state. Deep links work. Server Components re-render on URL changes.
8. **Audit + RBAC are invisible to the UI.** Every table write is caught by `audit.log_change()`; the UI never calls audit APIs directly. Permission checks happen server-side; the UI mirrors them for hide/show decisions but does not trust them.

---

2. Page map

2.1 Provider surface

Route	Purpose	Primary role(s)	Shipped?
`/`	Today dashboard (encounters count, unsigned notes, recent patients)	physician, nurse_ma	✅ Slice 1 (stub data)
`/patients`	Search + paginated patient list	all clinical + biller	✅ Slice 1
`/patients/new`	Create a new patient chart	scheduler, practice_owner, office_admin, physician, nurse_ma	✅ Slice 1
`/patients/[id]`	Chart — demographics + contacts + coverages	all (read); write per role	✅ Slice 1 + coverage PR
`/appointments?view=list&date=YYYY-MM-DD`	Day-view list	scheduler, office_admin, providers	✅ Slice 2
`/appointments?view=week&date=YYYY-MM-DD`	**Week-grid calendar**	same	🆕 this PR
`/appointments/new?date=&patient_id=`	Book appointment	scheduler, office_admin, owner	✅ Slice 2
`/appointments/[id]`	Detail + status transitions + Open encounter	same	✅ Slice 2 / Slice 3
`/encounters`	Provider-scoped encounter list	clinical roles	✅ Slice 3
`/encounters/[id]`	Encounter workspace (summary, diagnoses, vitals, SOAP, sign, amend, history)	clinical roles	✅ Slice 3 / 4 + diagnoses PR

2.2 Admin surface

Route	Purpose	Shipped?
`/admin`	Tenant overview / settings	✅
`/admin/members`	Invite/manage staff	✅
`/admin/payers`	Manage insurance companies	✅
`/admin/billing`	Billing dashboard	❌ stub
`/admin/security`	Audit log viewer	❌ TBD
`/admin/settings`	Tenant settings	❌ TBD

2.3 Patient surface

Route	Purpose	Shipped?
`/my`	Patient dashboard	✅ (stub)

`/my/appointments`	Upcoming/past appointments	❌ TBD
`/my/records`	Shared records	❌ TBD

2.4 Auth surface

Route	Purpose	Shipped?
`/login`	Email + password sign-in	✅
`/forgot-password`	Reset flow	✅
`/reset-password`	New password post-email	✅
`/set-password`	Invite-accept + password	✅
`/auth/callback`	Supabase code exchange	✅

---

3. Route structure

Next.js App Router with three route groups (parentheses = route group, doesn't segment URL):

```
apps/web/src/app/
├── (app)/                    # Authenticated shell: sidebar + topbar
│   ├── layout.tsx            # getSession() + redirect to /login if null
│   ├── page.tsx              # / — dashboard
│   ├── _shell/app-shell.tsx  # client shell (sidebar/topbar/theme toggle)
│   ├── patients/
│   │   ├── page.tsx          # list
│   │   ├── new/page.tsx      # create
│   │   └── [id]/page.tsx     # chart
│   ├── appointments/
│   │   ├── page.tsx          # list OR week view (?view=list|week)
│   │   ├── new/page.tsx      # booking
│   │   └── [id]/page.tsx     # detail
│   ├── encounters/
│   │   ├── page.tsx          # list
│   │   └── [id]/page.tsx     # workspace
│   ├── admin/
│   │   ├── layout.tsx        # admin-surface guard
│   │   ├── page.tsx
│   │   ├── members/page.tsx
│   │   ├── payers/page.tsx
│   │   └── …
│   └── my/                   # patient self-serve
├── (auth)/                   # Unauthenticated shell: centered card
│   ├── layout.tsx
│   ├── login/page.tsx
│   ├── forgot-password/page.tsx
│   ├── reset-password/page.tsx
│   └── set-password/page.tsx
└── auth/
    └── callback/route.ts     # not a page; code-for-session exchange
```

URL conventions

- List views: `?page=N&limit=M&q=…&status=…`
- Date-scoped views: `?date=YYYY-MM-DD`
- View toggle: `?view=list|week|month`
- Preset from another view: `?patient_id=…`, `?encounter_id=…`
- Flash status: `?ok=…` / `?error=…` (URL-encoded short message) — cleared on next navigation

No client-side navigation state. Every reload / share of the URL reproduces the view.

---

4. Component hierarchy

4.1 Shell (shared across all app pages)

```
<AppLayout> (app/(app)/layout.tsx)
  <AppShell>                 (client, _shell/app-shell.tsx)
    <TopNav />               (surface switcher, user menu, impersonation banner)
    <Sidebar>                (nav items filtered by permissions)
      <NavSection>
        <NavItem />
    <main>
      <ImpersonationBanner?/>
      {children}             ← page content
```

Permission-aware filtering: `surfacesFor(userKind, roles)` produces `["provider","admin"]` etc.; the TopNav renders those as tabs and filters the Sidebar's nav items by `requires: Permission[]`.

4.2 Page-level building blocks

```
<Page>
  <AppBreadcrumbs items={[...]} />
  <PageHeader eyebrow title description actions />
  <FlashBanner ok={sp.ok} error={sp.error} />     ← conditional, built inline today
  <main>
    <Card>                                          (one per logical section)
      <CardHeader><CardTitle /></CardHeader>
      <CardContent>
        [read-only view OR <form action={serverAction}>]
      </CardContent>
    </Card>
  </main>
```

4.3 Patient chart

```
/patients/[id]/page.tsx (Server Component)
├── Header: name, MRN, age, sex, pronouns, deceased badge
├── Card: Demographics             (editable inline w/ patient:write)
├── Card: Contacts                 (add inline + per-row remove)
└── Card: Coverages                (payer-picker add form + per-row toggle)
```

Rendering rules:

- Every card has both the read-only display and the editable form — which one renders is decided by `canWrite = session.permissions.includes("patient:write")`.
- For scheduler (`patient:demographics_only`, not yet filtered), the page renders the same data; a future refinement narrows the visible columns.

4.4 Appointment list / calendar

```
/appointments/page.tsx (Server Component, takes ?view=list|week)
├── Filter bar: date picker, status select, view toggle, prev/today/next
├── if view=list:
│   └── Card: day list → Table(time / patient / provider / reason / status)
└── if view=week:
    └── Card: WeekGrid
        └── Grid: 7 day columns × N time rows, absolute-positioned cells
            - Click appointment → /appointments/[id]
```

Week grid is a **pure server render** (CSS-grid layout) — no client JS needed. Sub-hour appointments use explicit inline grid-column / grid-row styling.

4.5 Appointment detail

```
/appointments/[id]/page.tsx
├── Header: patient name, MRN, time, provider, location, status badge
│   actions: Open encounter, Open chart
├── Card: Quick-action status transitions (gated on schedule:write + active status)
├── Card: Visit details (editable form: time, duration, reason, visit type)
├── Card: Cancelled (read-only, shows when status=cancelled)
├── Card: Cancel appointment (form, gated + active)
└── Card: Encounter (link if encounter_id set, instruction otherwise)
```

4.6 Encounter workspace

```
/encounters/[id]/page.tsx
├── Header: patient, provider, start time, status badge
├── Card: Visit summary (editable: chief_complaint, status, reason)
├── Card: Diagnoses (ICD-10 list + add form)
├── Card: Vitals (timeline + entry form)
├── Card: Clinical note
│   ├── Signed: read-only view + Amend form (if clinical:amend)
│   └── Draft: editor + "Ready to sign" form (if clinical:sign)
└── Card: Version history (visible once v2+ exists)
```

---

5. Data-fetching strategy

Need	Mechanism
List / detail reads	Server Component runs Supabase query in render; result is the page. Uses the authenticated cookie session; RLS filters per-tenant.
Mutations	Server Actions (`"use server"`) inside the page file, submitted via `<form action={...}>` . Redirect back to the same URL with `?ok=` / `?error=` . `revalidatePath()` flushes the cache.

Cross-page revalidation	Explicit `revalidatePath("/patients")` after mutations that might change a list upstream.
Cache	Page-level `export const dynamic = "force-dynamic"` on every authenticated page — session-dependent data must not be cached.
Concurrent queries	Page-level Promise.all not yet needed; each query is cheap and serial is fine. Upgrade when a page crosses ~4 queries or 300ms.
Pagination	URL param `page` + Supabase `.range()`; COUNT returned via `{ count: 'exact' }`. Default 25/page, hard cap 100.
Search	Server-side `ilike '%q%'` with `.or()` across relevant columns. Full-text is deferred until row count warrants it (100k+).
Relations	Nested select syntax — `patient:patient_id(given_name, family_name, mrn)`. Cast with `as unknown as` because the generated `Database` types don't reinfer through `@supabase/ssr`.
Optimistic UI	Not used — form submits round-trip. Fine for an enterprise app where confirmation is expected.

Example (today's /patients/page.tsx):

```
let query = supabase
  .from("patients")
  .select("...", { count: "exact" })
  .eq("tenant_id", session.tenantId)
  .is("deleted_at", null)
  .range(from, to);
if (q) query = query.or(`family_name.ilike.%${q}%,…`);
const { data, count, error } = await query;
```

---

6. Permissions by page / action

All pages run `requirePermission(session, key)` at the top; actions inside run their own `requirePermission` before the mutation. UI rendering hides inaccessible sections but never trusts itself — the server is authoritative.

Surface / action	V1 permission
Patients	
View list / chart	`patient:read`
Create patient	`patient:write`
Edit demographics	`patient:write`
Add/remove contact	`patient:write`
Add/edit coverage	`patient:write`

Appointments	
View list / week / detail	`schedule:read`
Create / edit / cancel	`schedule:write`
Status transition	`schedule:write`
Open encounter	`clinical:write` (creates the encounter; `schedule:write` enough to link existing)
Encounters	
View list / workspace	`clinical:read`
Edit summary / draft	`clinical:write`
Record vitals	`patient:write`
Assign / remove diagnosis	`clinical:write`
Sign note	`clinical:sign`
Amend note	`clinical:amend`
Admin	
Members	`admin:users`
Payers	`admin:billing_config`

Hiding patterns in the page:

```
const canWrite = session.permissions.includes("patient:write");
{canWrite ? <form action={updatePatient}>…</form> : <dl>{readOnly}</dl>}
```

For future V2 migration to the module-level matrix (`patient_records:update` , `encounters:sign` , …), these checks will swap 1:1 with the map in docs/permissions-matrix.md.

---

7. Validation and form flows

7.1 Validation layers (in order)

1. **HTML form attributes** — `required` , `type=email` , `minLength` , `maxLength` , `pattern` . Free.
2. **Server action top-of-function checks** — explicit `if (!email.includes("@"))` style guards before the DB call. Redirects with `?error=...` .
3. **Zod schemas** (in packages/types/src/clinical/*) — for when a field has non-trivial rules (ICD-10 regex, date range, SSN-last-4). Not yet plumbed into the server actions; follow-up.

4. **Database CHECK constraints** — ultimate backstop (e.g. `attachments_kind_check`, `diagnosis_assignments.rank between 1 and 12`). Errors surface as Postgres messages.

7.2 Form submission flow

```
User submits <form action={serverAction}>
    │
    ▼
Server action runs (edge or node, user's session in cookies)
    ├── getSession() → null? → redirect /login
    ├── requirePermission(…) → throw 403 (handled by error boundary)
    ├── validate inputs → invalid? → redirect(?error=…)
    ├── Supabase mutation (RLS applies)
    │       └── DB error? → redirect(?error=…)
    ├── revalidatePath(…)
    └── redirect(?ok=…)
```

No client-side JS needed for any of the existing forms — plain HTML forms + Server Actions. That's deliberate: robust against network issues, degrades cleanly without JS, no client-bundle weight.

7.3 Feedback pattern

- Success → `?ok=<message>` in URL → green banner at top of page.
- Error → `?error=<message>` → red banner.
- Banners auto-clear on next navigation.
- Destructive actions (remove member, cancel appointment, write-off) require a reason (textarea) + server-side length check.

7.4 Concurrency

Current state: **last-write-wins**. Acceptable for V1 with ~10 users per tenant. Before scaling:

- Add `updated_at` optimistic lock on `encounter_notes` drafts (already called out in note.ts:53 as `ifUnmodifiedSince`).
- On conflict, re-render the form with the server's current content + a merge banner.

---

8. Edge cases

Case	Handling
Patient MRN collision on create	DB unique constraint rejects; server action shows the Postgres error verbatim (acceptable for V1).
Appointment outside business hours	Allowed — enterprise practices run 24/7 in ED contexts.

Appointment end before start	Server action checks; returns `?error=invalid time`.
Provider double-booking	Soft warning in V1 (none today); hard exclusion constraint in V2.
Deleting a patient with active encounters	Blocked at DB via FK + RLS; UI doesn't expose delete yet.
Navigating to a patient you can't see	RLS returns null → page calls `notFound()` → 404.
Signed note, user refreshes	Page shows signed read-only view; Amend form gated by `clinical:amend`.
Amendment reason too short	Server action checks ≥5 chars; returns `?error=…`.
Session expired mid-form	Action redirects to `/login?next=<current-path>`.
Concurrent edits on same draft	Last-write-wins; V2 introduces optimistic lock.
Cancelled appointment + accidental rebook on same date	Allowed — two rows, first is terminal, second is active. Flagged in UI only via filter.
Timezone mismatch (user in EDT, data in UTC)	Day-view filter uses UTC for now; see Slice 2 caveat. Week view same. Fix: `locations.timezone` wiring.
`clinical:sign` user impersonating	`IMPERSONATION_BLOCKED` strips `clinical:sign` — the Sign button is hidden and the server action refuses.
Browser back after form submit	Redirect-to-GET pattern avoids POST resubmission.
Calendar week spanning a DST change	Grid rendered in UTC; hour labels local to browser. Still correct; no lost hours.

---

9. Acceptance criteria

9.1 Patient search / list

- Typing a name substring and pressing Enter filters the list server-side.
- MRN search is exact + partial.
- Empty state appears when no rows; CTA to create.
- Pagination at 25/page; URL preserves page/q.
- `patient:read` role sees the list; other roles 404 on direct navigation.

9.2 Patient profile

- Direct navigation to a patient id in another tenant → 404.
- Demographics card editable when `patient:write`; read-only otherwise.
- Contacts inline add/remove; primary badge on the primary contact of each type.
- Coverages card lists all coverages (active + inactive); add form hidden when no active payers exist, with link to `/admin/payers`.
- Coverage activation toggle flips `active`; DB audit event present.

9.3 Patient insurance summary

- Coverages section on `/patients/[id]` as above.
- Payer picker populated from `public.payers WHERE active = true`.
- Primary / secondary / tertiary / self-pay / workers-comp / auto / other selectable.
- Deactivated coverage still visible (read-only) with muted badge.

9.4 Appointment list / calendar

- `?view=list` — day list, prev/today/next navigation.
- `?view=week` — 7-column grid of a given week, appointments block-rendered by start/duration.
- Status filter applies to both views.
- Empty day → EmptyState with CTA to create.
- Click an appointment block → `/appointments/[id]`.
- Permission-gated; `schedule:read` reads, `schedule:write` writes.

9.5 Appointment detail

- Header shows patient + provider + time + location + status.
- Status quick-action buttons respect APPOINTMENT_STATUS_TRANSITIONS.
- Cancel requires a reason (min 1 char, max 1024).
- "Open encounter" creates an encounter tied to the appointment (if none) and navigates; otherwise links.

9.6 Encounter launch

- Accessible from appointment detail when `schedule:read + clinical:write`.
- Creates `public.encounters` row with class= `ambulatory`, status= `in_progress`, linking `appointments.encounter_id`.
- Idempotent: a second click (after the first succeeds) navigates to the existing encounter instead of creating another.
- Button hidden for `cancelled` / `no_show` / `completed` appointment states.

---

10. What's new in this PR

1. **Week-grid calendar view** on `/appointments` — toggle `?view=week`. Implementation in apps/web/src/app/(app)/appointments/page.tsx. 7 columns × hourly rows (08:00–19:00), appointments block-rendered with absolute positioning inside a CSS grid.
2. This design doc.

---

11. Deferred

- **Patient typeahead** on appointment-new — today is MRN lookup only. Client component + search endpoint.
- **Month calendar view** — less useful than week for scheduling; deferred.
- **Drag-to-reschedule** — material client-side lift; deferred.
- **Demographic-only scheduler view** of patient chart — narrows columns when role is `scheduler`. 30-line change once V2 permissions land.
- **Patient portal (`/my/*`)** — patient-kind surface mostly stubs today.
- `/admin/security` **audit log viewer** — scoped separately.